MIND MATRIX

NAME: MOHAMMED SAHIL R

EMAIL ID: mohammedsahilr39@gmail.com

SESSION: 07

REPORT ON: TASK REPORT

TASK NO: 01

TITLE:

STATE CONCEPTS DEEP DIVE & VISUAL MAPPING

Objective

The objective of this task is to understand how state works in Jetpack Compose and how it controls UI updates. This includes learning key concepts such as remember, mutableStateOf, recomposition, and state hoisting.

Key Concepts

- **State**
  State represents the current data that determines what is displayed on the UI. When state changes, the UI updates automatically.

Example:

```
var count by remember { mutableStateOf(0) }
```

- **remember()**
  The remember function stores values across recompositions so that they are not recreated every time the UI refreshes.

Example:

```
val name = remember { mutableStateOf("") }
```

- **mutableStateOf()**
  This function creates a state holder that Compose observes. When the value changes, the UI recomposes.

Example:

```
var text by remember { mutableStateOf("") }
```

- **State Hoisting**
  State hoisting means moving state from a child composable to a parent composable so the child becomes reusable and stateless.

- **Recomposition**
  Recomposition occurs when Compose redraws the UI because the state has changed.

Interaction Scenario

Example scenario: Tip Calculator

User enters bill amount → tip is calculated → user toggles round-up switch → result updates.

Flow:

User Input → State Variable → UI Update

Bill amount input → billAmount state → result displayed

Switch toggled → roundUp state → recalculated tip shown

State Handling Table

UI Component	State Variable	Stateful or Stateless	Where State Defined	Comments
TextField	billAmount	Stateful	Main Screen	Stores user input
Switch	roundUp	Stateful	Main Screen	Controls rounding
Text	result	Stateless	Derived state	Displays calculated result

Reflection

State hoisting helps maintain clear separation between UI components and data management. If state is deeply nested in child composables, it becomes difficult to manage updates and debugging. One possible bug is when toggling a switch but the UI does not update because the state is stored incorrectly.